Advanced Cryptography Elisabeth Oswald

Lab Sheet 2

2025 Formative Exercises

Introduction

This lab sheet supports your learning for the second "half" of the module (i.e. the content that we go through until the end of W09). All of these exercises are *formative*: this means that you can work on them by yourself, with a partner, or even in groups. There are no grades available for these, and you are encouraged to discuss your solution with the other students, or, if you continue to have doubts/questions, with a teaching assistant and/or a lecturer.

Types of exercises. There are a number of different tasks that develop different skills. Some tasks require you to do something with "paper and pencil", some tasks require perhaps a bit of further reading/research, and some require to implement something in Python.

Python

If you are experienced with Python then use whatever environment you like to work with; just ensure that when it comes to the coursework submission, your code happily runs with Python 3.

If you are not so familiar with Python, then I recommend to use PyCharm Community by JetBrains: `https://www.jetbrains.com/pycharm/` (free to use, the commercial version is also free for students but you must provide proof of student status). The main reason for this recommendation is that it offers a reasonably easy to use IDE, and you can set up virtual environments for different projects. If you do so, then, if you mess up one VE with libraries etc., you only mess up one project.

1 DPA basic technique

Experiment with the provided Python code provided in the file `dpa_demo.py` that perform differential input attacks. Start off with the data in 'WS1.h5'. This file is a 'HDF5'

database. It enables a simple way of storing multiple datasets in a structured way. You will have to install the `h5py` Python library to work with it. Also in order to use various numpy functionality you need the Python `numpy` library , and in order to plot, you need the Python `matplotlib` library.

An HDF5 file stores datasets and groups. Groups work like dictionaries, so when you open an .h5 via `f = h5py.File('name')` you can then use `f.keys()` to see what datasets are in the "main group". I provide you with code that opens and extracts the available data from each provided dataset. So you should not need to mess around with the .h5 files much.

The dataset in 'WS1.h5' is very similar to the dataset that I used to produce the pictures that you see in the slides and the notes. It only represents a short snippet (of longer measurements) that focuses on the `AddRoundKey` and `SubBytes` operation of a single state byte.

Run `dpa_demo.py` and play around with the different configurations within the implementation of the correlation distinguisher (i.e. the `PCor()`) function.

- Change the intermediate value from `SubBytes` to `AddRoundKey`. Revisit the question why the latter does not give "as good results" (in what way are they not "as good" and why is this the case)?
- Change the power model to the LSB model, and compare the outcome of the `PCor()` attack with the outcome of the `DoM` attack. Do they look similar or different to you? If you think that they are similar, try to explain why this makes sense.

2 A slightly more advanced DPA

Now move on to working with the data in `AT89_SubBytes.h5`. The data in there is again from the same implementation as the data in `WS1.h5`, but I provide more data for you to play with. There is code in the demo script already to open and read in data from this file.

First of all, run the correlation and Dom distinguisher to get a successful key recovery attack working. Then, using the "engineering approach" to working out the number of needed traces for a successful attack, produce a "evolution" plot like I provide in the slides/notes. This means that for sets of traces, whereby the number of traces per set increases, run attacks to determine the maximum correlation/DoM value for correct/incorrect key hypothesis and some time point ct, and plot this data. You should produce a plot that has the number of traces per attack along the x-Axis, and the correlation/Dom values along the y-Axis. Given the best estimate r that you have for ρ, check how well

this corresponds to the table in the notes that give NNT estimates for various values of ρ. Is there a good correspondence (why/why not)?

I also provide functionality to compute an SNR trace. Try to get this to work for this dataset. Then produce SNR traces for an adversary that uses

1. the LSB power model
2. the HW power model
3. an "identity power model" (i.e. each intermediate value gets its own class).

Try and make sense of the resulting SNR plots. Why do some show a higher SNR than others? Are plots are based on estimating the SNR: are some estimates better than others? Why? Do the SNR plots "align" with the results from the correlation/Dom attacks?

3 Side channel simulations: attacking $(2, 2)$ secret sharing

The Python code contains a simulation that shows an attack on a $(2, 2)$ Boolean secret shared SubBytes lookup. The simulation is such that the output of SubBytes is represented as $S(a) = (a_1, a_2)$, whereby a_1 is random byte value and $a_2 = S(a) \oplus a_1$. The simulation assumes that an adversary sees leakage on a_1 and a_2. The simulated traces contain one further trace point that is based on independent noise only. The secret key is randomly sampled at the beginning of the simulation, alongside the input data (the same key value is applied to all input data).

The attack that is implements is based on pre-processing traces to create joint leakage on a_1 and a_2. This is done via the "mean-free product" method that we talk about in the lectures. Once this pre-processing is done, the new traces (called `prod_traces` in the code), can be subjected to a conventional differential input attack. The script shows that key recovery works.

Extend/modify this simulation so that you can run attacks for the following implementation options:

1. leakage on the input and output of a (masked) table look-up is available to the adversary, and input and output mask are identical
2. different input and output masks are used for the table look-up, and leakage on both masks is available

3. different input and output masks are used for the table look-up, but the masks never leak as such; however as all state bytes use the same randomness the leakage of multiple table look-ups can be utilised

4 Provable security

We looked at different ways to compute a multiplication of two values that are secret shared. In particular, when assuming $(2, 2)$ Boolean secret sharing, the multiplication is based on computing and summing the partial products:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_1b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_2b_2$

We observed in the lectures that this particular way of summing up the partial products is insecure, and then showed, by changing what we sum up and introducing extra, fresh randomness, how to do a secure version.

Consider another alternative way of summing up the partial products:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_2b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_1b_2$

Here, the first output share is the sum over the diagonal, and the second output share sums the terms off the diagonal.

Show that this way of computing the secret shared product is also not secure:

1. Show, theoretically, using any of the strategies to reason about the security of secret shared computations, that this computation will leak information about either a or b or both.

2. Give a practical demonstration of an attack on this multiplication by writing a simulation. Your simulation should take the secret values a, b as inputs, as well as the number of power traces for the attack. Simulate leakage for the intermediate values that will occur (leak the HW of intermediate values, you can elect to not add any noise, so produce a best case simulation). You can elect to choose a to be a bit value or a byte value. If you go for the latter option, then use the `Galois` library (`pypi.org/project/galois`) to implement the finite field multiplication.

5 Design/Assess countermeasures for DES

You have been head-hunted by a large car manufacturer. Because of disastrous press on their existing key fob system [1], they now wish to deploy strong cryptography, and at the same time, they wish to secure they wireless key fob against side channel attacks. As they are a "forward looking" company, they decided that it is time to (at least) upgrade to using the DES algorithm. They need your expertise to make an informed choice about the implementation of countermeasures.

DES was designed specifically for hardware implementations, and can run in very resource constraint environments. It has a small state (64 bits). At the beginning of a DES round, half of the state is exclusive-ored with a round key (akin to what happens in the AES `AddRoundKey` function). Then the state is expanded and 8 different substitutions $S_i()$ are applied to six bits of the state. The substitutions are "compressing" and give a four bit output. Then the state bits are permuted again. The whole process is depicted inFig. 1.

1. Explain the five steps for a differential input attack.
2. Reason what side channels are a particular threat to a key fob.
3. What type of secret sharing would be most compatible with DES?
4. DES has eight different substitution boxes, and, it does bit-wise permutations. What hiding strategies would you recommend ?
5. The company also approached you with several questions, where they wish clarification:

   A. They heard that you need at least a $(3, 3)$ Boolean secret sharing scheme to achieve security against differential input attacks. Is this true? What security guarantee(s) does a $(3, 3)$ Boolean secret sharing scheme give?

   B. They are aware of the $3 - DES$ construction (i.e. doing DES three times with either two or three keys), and wonder if $3 - DES$ would increase the number of needed traces for a differential input attack by a factor of 3?

   C. They also consider strengthening DES by running it with more rounds. Would this make the algorithm harder to attack using side channels?

[1] `https://www.esat.kuleuven.be/cosic/news/fast-furious-and-insecure-passive-keyless-entry-and-star`

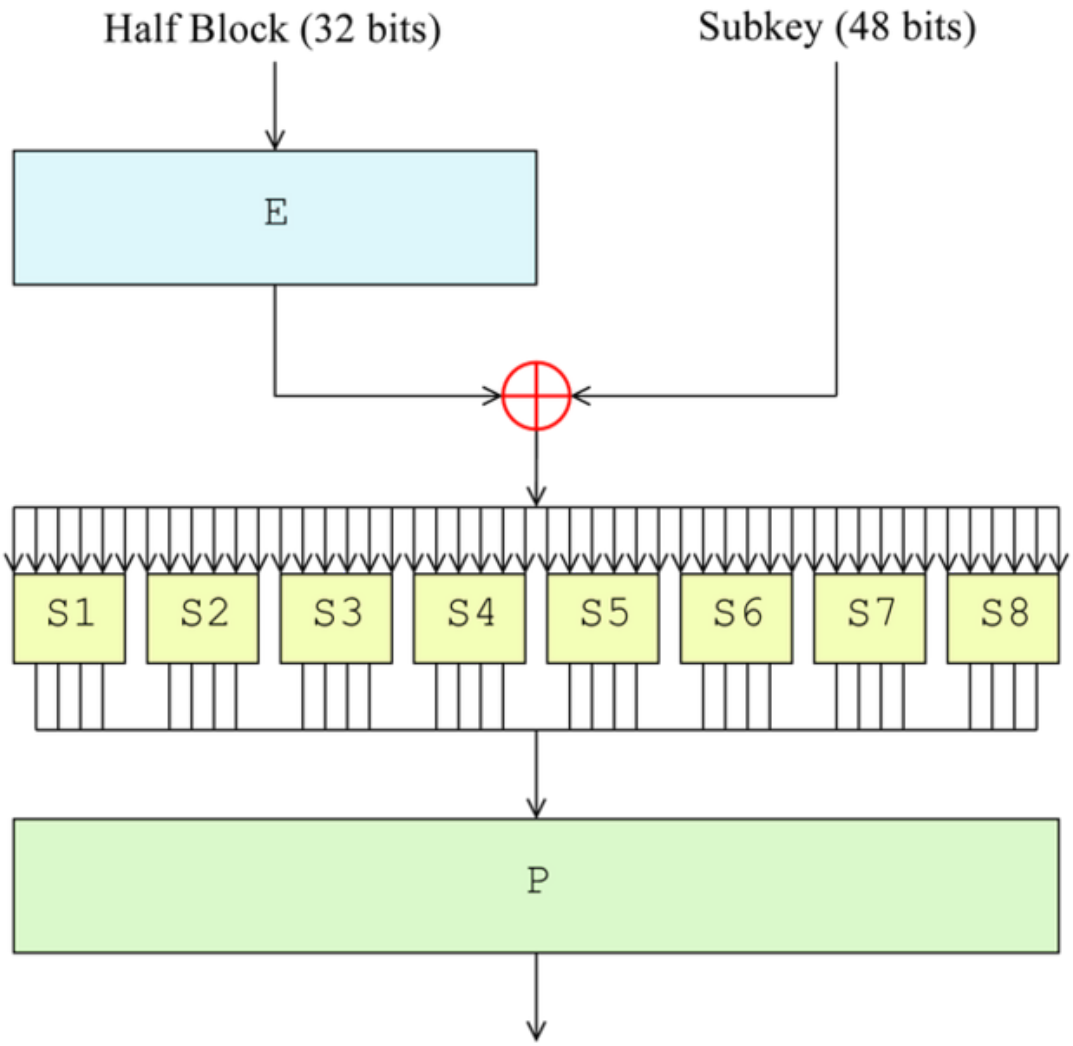

Figure 1: DES round